

```
CREATE DATABASE almacenes3;  
DROP DATABASE almacenes3;  
use almacenes3;
```

```
CREATE TABLE cliente (  
  ClienteID INT PRIMARY KEY,  
  Nombre VARCHAR(30),  
  Dirección VARCHAR(50),  
  Telefono VARCHAR(15))ENGINE=INNODB;
```

```
CREATE TABLE producto (  
  ProductoID INT PRIMARY KEY,  
  Nombre VARCHAR(30),  
  Descripcion VARCHAR(50),  
  Precio_coste FLOAT,  
  PVP FLOAT,  
  Stock INT)ENGINE=INNODB;
```

```
CREATE TABLE factura (  
  FacturaID INT PRIMARY KEY,  
  ClienteID INT,  
  Fecha_Emission DATE,  
  Descuento FLOAT,  
  FOREIGN KEY (ClienteID) REFERENCES cliente(ClienteID)  
  ON DELETE CASCADE ON UPDATE CASCADE)ENGINE=INNODB;
```

```
CREATE TABLE lineas_factura (  
  FacturaID INT,  
  LineaID INT,  
  ProductoID INT,  
  cantidad FLOAT,  
  PRIMARY KEY (FacturaID, LineaID),  
  FOREIGN KEY (FacturaID) REFERENCES factura(FacturaID)  
  ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (ProductoID) REFERENCES producto(ProductoID)  
  ON DELETE CASCADE ON UPDATE CASCADE)ENGINE=INNODB;
```

```
CREATE TABLE proveedor (  
  ProveedorID INT PRIMARY KEY,  
  Nombre VARCHAR(50),  
  Direccion VARCHAR(50),  
  Telefono VARCHAR(15))ENGINE=INNODB;
```

```
CREATE TABLE suministro (  
  SuministroID INT PRIMARY KEY,  
  ProveedorID INT,  
  Fecha_emision DATE,  
  FOREIGN KEY (ProveedorID) REFERENCES proveedor(ProveedorID)
```

```
ON DELETE CASCADE ON UPDATE CASCADE)ENGINE=INNODB;
```

```
CREATE TABLE linea_suministro (  
SuministroID INT,  
LineaSuministroID INT,  
ProductoID INT,  
cantidad FLOAT,  
PRIMARY KEY (SuministroID, LineaSuministroID),  
FOREIGN KEY (SuministroID) REFERENCES suministro(SuministroID)  
ON DELETE CASCADE ON UPDATE CASCADE,  
FOREIGN KEY (ProductoID) REFERENCES producto(ProductoID)  
ON DELETE CASCADE ON UPDATE CASCADE)ENGINE=INNODB;
```

```
-- CLIENTE
```

```
INSERT INTO cliente VALUES  
(1,'Ambrosio Casado','Calle Principal, Ciudad','123-456-7890'),  
(2,'Amor Rodríguez','Avenida Secundaria, Ciudad','987-654-3210'),  
(3,'Heraclio Borbujo','Calle Secundaria, Pueblo','555-123-4567'),  
(4,'Meli Ramos','C/Los Sauces','980-675-897');
```

```
Select * from cliente;
```

```
-- PRODUCTO
```

```
INSERT INTO producto VALUES  
(101, 'Cemento', 'Tipo A, 50 kg',5, 8.50, 100),  
(102, 'Ladrillos', 'Rojos, Paquete de 100',10.00, 20.00, 200),  
(103, 'Madera', 'Pino, 2x4, 8 pies',3.0, 7.50, 150),  
(104, 'Tornillos', 'Acero inoxidable, 50 unidades', 2.0,5.00, 300),  
(105, 'Bañeras de Hidromasaje', 'Porcelana fina', 540.0,125000, 20);
```

```
Select * from producto;
```

```
-- FACTURA
```

```
INSERT INTO factura VALUES  
(501, 1, '2024-01-08', 0),  
(502, 1, '2024-01-09', 2),  
(503, 1, '2024-01-10', 7),  
(504, 2, '2024-01-11', 20),  
(505, 2, '2024-01-12', 0),  
(506, 3, '2024-01-13', 10),  
(507, 4, '2024-01-13', 15);
```

```
Select * from factura;
```

```
-- LINEAS DE FACTURA
```

```
INSERT INTO lineas_factura VALUES
```

```
(501, 1, 101, 5),
(501, 2, 102, 2),
(502, 1, 103, 3),
(503, 1, 104, 4),
(503, 2, 101, 2),
(504, 1, 102, 3),
(505, 1, 103, 2),
(505, 2, 104, 1),
(506, 1, 101, 4),
(506, 2, 103, 1),
(507,4,105,1);
```

```
Select * from lineas_factura;
```

```
-- PROVEEDOR
```

```
INSERT INTO proveedor VALUES
```

```
(1, 'Proveedor X', 'Calle Principal, Ciudad', '111-222-3333'),
(2, 'Proveedor Y', 'Avenida Secundaria, Ciudad', '444-555-6666');
```

```
Select * from proveedor;
```

```
-- SUMINISTRO
```

```
INSERT INTO suministro VALUES
```

```
(701, 1, '2024-01-15'),
(702, 2, '2024-01-16'),
(703, 1, '2024-01-17');
```

```
Select * from suministro;
```

```
-- LINEAS SUMINISTRO
```

```
INSERT INTO linea_suministro VALUES
```

```
(701, 1, 104, 10),
(702, 1, 101, 15),
(703, 1, 102, 5),
(701, 2, 103, 4),
(702, 2, 101, 5),
(702, 3, 102, 10),
(703, 2, 103, 3),
(703, 3, 105, 4);
```

```
Select * from linea_suministro;
```

```
alter table producto add Stock_min int;
```

```
update producto set Stock_min=10 where ProductoID=101;
```

```
update producto set Stock_min=100 where ProductoID=102;
```

```
update producto set Stock_min=50 where ProductoID=103;
```

```
update producto set Stock_min=250 where ProductoID=104;
update producto set Stock_min=7 where ProductoID=105;
```

```
-- Creación de la tabla pedir
create table if not exists pedir(
ProductoID int,
fecha date,
Stock int)Engine=innodb;
```

```
-- TRIGGER ----
```

```
DELIMITER $$
create trigger actualizarStockAI after insert on lineas_factura for each
row begin
if new.cantidad>0 then
update producto p set Stock=Stock-new.cantidad
where new.ProductoID=ProductoID;
else update producto set Stock=Stock+new.cantidad
where new.ProductoID=ProductoID;
end if;
```

```
-- insertar en la tabla pedir, el producto a pedir
insert into pedir (ProductoID, Fecha, Stock)
select new.ProductoID, NOW(), Stock
from producto
where ProductoID = new.ProductoID
and Stock <= Stock_min;
```

```
end $$
delimiter ;
DELIMITER $$
create trigger actualizarStockAI2 after insert on lineas_factura for each
row begin
update producto p set Stock=Stock-new.cantidad
where new.ProductoID=ProductoID;
```

```
-- insertar en la tabla pedir, el producto a pedir
insert into pedir (ProductoID, Fecha, Stock)
select new.ProductoID, NOW(), Stock
from producto
where ProductoID = new.ProductoID
and Stock <= Stock_min;
```

```
end $$
delimiter ;
```

```

drop trigger actualizarStockAI;

-- correccion para reducir el stock
DELIMITER $$
create trigger reducirStockAI after insert on lineas_factura for each row
begin
update producto p set Stock=Stock-new.cantidad
where new.ProductoID=ProductoID;

end $$
delimiter ;
DELIMITER $$
create trigger reducirStockAI after insert on lineas_factura for each row
begin
update producto p set Stock=Stock+new.cantidad
where new.ProductoID=ProductoID;

end $$
delimiter ;

-- insertar lineas de factura. para insertar en lineas facturas hay que
insertar primero en factura
INSERT INTO factura VALUES
(508, 1, '2024-01-10', 0);

INSERT INTO factura VALUES
(509, 2, '2024-01-10', 0);

INSERT INTO lineas_factura VALUES
(508, 1, 101, 10),
(508, 2, 102, 50);

INSERT INTO lineas_factura VALUES
(509, 1, 101, 85),
(509, 2, 103, 102);

INSERT INTO factura VALUES
(510,1, '2024-01-10', 0);

INSERT INTO lineas_factura VALUES
(510, 1,101, -5);

-- Prueba de función para el stock bajo
DELIMITER $$
CREATE FUNCTION es_stock_bajo(producto_id INT)
RETURNS BOOLEAN
BEGIN
DECLARE stock_actual INT;

```

```

DECLARE stock_minimo INT;
-- Obtener el stock actual y el stock mínimo del producto
SELECT Stock, Stock_min INTO stock_actual, stock_minimo
FROM producto
WHERE ProductoID = producto_id;
-- Devuelve true si el stock es bajo, false en caso contrario
RETURN stock_actual <= stock_minimo;
END$$
DELIMITER ;

-- Prueba procedimiento para recibir mensaje en caso de stock bajo
DELIMITER $$
CREATE PROCEDURE mensaje_stock_bajo(IN producto_id INT)
BEGIN
DECLARE mensaje VARCHAR(100);
DECLARE stock_actual INT;
DECLARE stock_minimo INT;
-- Verificar si el stock es bajo usando la función
IF es_stock_bajo(producto_id) THEN
-- Obtener el stock actual y el stock mínimo para el mensaje
SELECT Stock, Stock_min INTO stock_actual, stock_minimo
FROM producto
WHERE ProductoID = producto_id;
-- mensaje para stock bajo
SET mensaje = CONCAT('El producto con ID ', producto_id, ' tiene un stock
bajo. Stock actual: ', stock_actual, '. Stock mínimo: ', stock_minimo);
ELSE
-- mensaje si el stock no es bajo
SET mensaje = CONCAT('El producto con ID ', producto_id, ' tiene un stock
adecuado.');
```

END IF;

```

-- Mostrar el mensaje
SELECT mensaje AS 'Mensaje de Stock';
END$$
DELIMITER ;

-- aqui se llama al procedimiento
CALL notificar_stock_bajo(102);
```

```
DELIMITER $$
```

```
-- Función para verificar si el stock de un producto está por debajo del  
mínimo
```

```
CREATE FUNCTION VerificarStockMinimo(ProductoID INT)
```

```
RETURNS BOOLEAN
```

```
BEGIN
```

```
DECLARE stock_actual INT;
```

```
    DECLARE stock_minimo INT ; -- Puedes cambiar este valor según sea  
necesario
```

```
    -- Obtener el stock actual del producto
```

```
    SELECT Stock INTO stock_actual FROM producto WHERE ProductoID =  
ProductoID;
```

```
    -- Comparar el stock con el mínimo permitido
```

```
    IF stock_actual < stock_minimo THEN
```

```
        RETURN TRUE;
```

```
    ELSE
```

```
        RETURN FALSE;
```

```
        END IF;  
END$$
```

```
-- Procedimiento para verificar y mostrar un mensaje si el stock es bajo  
CREATE PROCEDURE ControlarStock(IN ProductoID INT)
```

```
BEGIN
```

```
    IF VerificarStockMinimo(ProductoID) THEN
```

```
        SELECT CONCAT('¡Atención! El producto con ID ', ProductoID, '  
tiene un stock bajo.') AS Mensaje;
```

```
    ELSE
```

```
        SELECT 'El stock es suficiente.' AS Mensaje;
```

```
    END IF;
```

```
END$$
```

```
DELIMITER ;
```